Readiness in Software Engineering

Improving project delivery, reducing waste, and increasing consistency.

Why Software Projects Struggle

Rework & Delays

Redoing work, delivery slips, quality issues.

Shifting Requirements

Scope changes mid-implementation.

Technical Debt

Compromised work slows teams.

Root cause: Premature starts without shared understanding.

What Is Readiness?

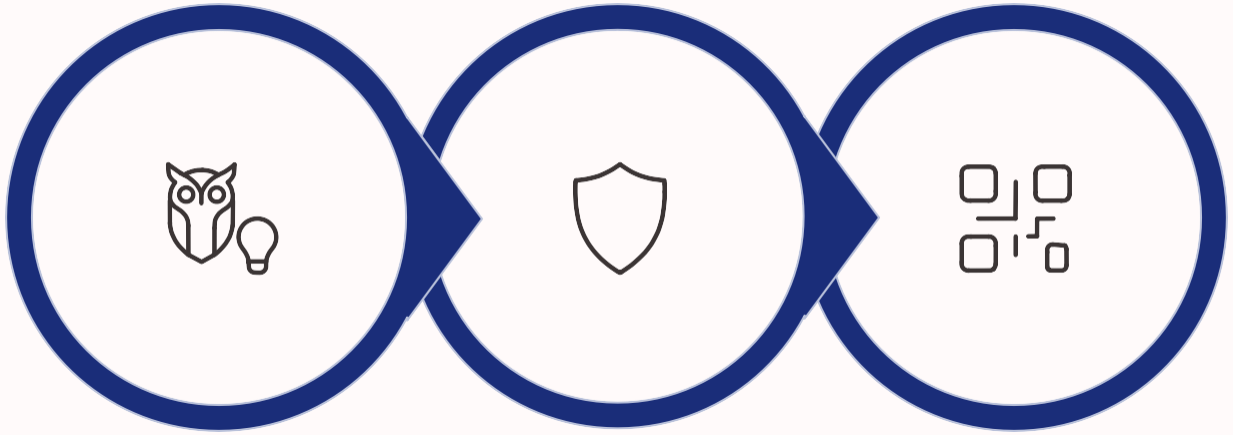
Understood
Work items are sufficiently understood.

Decomposed
Tasks are broken down.

De-risked
Ambiguity removed before development.

Prevents expensive waste: premature implementation.

Requirements Maturation Flow (RMF)



Understand **De-risk** **Code**

Operationalizes readiness, making it explicit in any lifecycle.

Moves problem-solving upstream, resolving vagueness early.

Three Foundational Practices

01

Shared Understanding

Align on intent, scope, and success criteria.

02

Definition of Ready (DoR)

Clear entry gate for development.

03

Definition of Done (DoD)

Final quality bar, tailored per task.

Definition of Ready (DoR)

A practical DoR ensures engineering puzzles are solved.

Key Checks:

- Technical Blueprints: Architecture cleared, API fit.
- UI/UX Mockups: Visuals clarify intent.
- Acceptance Criteria: Define feature behavior.

DoR: Beyond the Basics

1

Clarity

Unambiguous business intent, shared Definition of Success.

2

Decomposition

Small enough work, no hidden sub-tasks.

3

Technical Prerequisites

Environments, access, dependencies in place.

Readiness as Explicit Work

1. **Stop Premature Starts**

   If it isn't ready, don't start it.

2. **Eliminate Uncertainties Early**

   Resolve unknowns when cheap (on paper).

3. **Align Before Execution**

   Avoid mid-sprint arguments.

Investing in readiness clears the track for execution.

Conclusion

Software projects often fail from starting too soon.

> By making readiness explicit, teams reduce rework, avoid churn, and deliver consistently.

Less stress, higher quality.